

Informe - Desafío 2

UdeAWorldCup

Informatica II 2026-1

Antonio Carlos Merlano Ricardo

Introducción

El desafío quiere hacer una simulación de la copa del mundo. El programa tiene que leer una base de datos en Excel con 48 selecciones y sus jugadores, donde se realiza un sorteo de grupos con restricciones por la confederación, se simula la fase de grupos, también las fases eliminatorias y se determina el campeón. Al finalizar, actualiza los registros históricos en archivos de salida.

El desarrollo se enfocó en cumplir los siguientes requerimientos dados:

1. La gestión de la memoria dinámica.
2. Programación orientada a objetos.
3. Uso de arreglos y contenedores.
4. Simulación con probabilidades para resultados.

Análisis del problema y alternativa a solución

El problema que se tenía que resolver consistió en hacer una simulación de fútbol con condiciones: manejar 48 selecciones (cada una con 15 jugadores), realizar un sorteo por bombos, y simular tanto la fase de grupos (72 partidos) como las rondas eliminatorias. Además, era obligatorio actualizar los archivos históricos sin perder datos y programar contenedores de datos propios. También dividir el problema en clases e implementar estructuras dinámicas propias.

Se eligió la segunda opción por que al trabajar orientado a objetos, el código queda más ordenado y es más fácil identificar errores. También se permite separar la lógica matemática de los sorteos y los partidos, de las funciones que se encargan de leer y escribir los archivos.

Se divide en cuatro partes principales:

un Repositorio para el manejo de los datos y archivos, Torneo para administrar las reglas del campeonato, Simulador y Medidor de rendimiento.

Análisis de la solución propuesta

Requisitos funcionales

- Cargar equipos y jugadores desde archivos CSV.
- Sortear los grupos respetando bombos y confederaciones.
- Generar el calendario de la fase de grupos.
- Simular los 72 partidos correspondientes.
- Calcular la clasificación y determinar los mejores terceros.
- Estructurar las llaves de dieciseisavos, octavos, cuartos, semifinales, final y tercer puesto.
- Almacenar los resultados al finalizar la ejecución.
- Gestión explícita de la memoria dinámica.
- Separación clara de responsabilidades entre clases.
- Inclusión de diagnósticos mediante métricas de iteraciones y consumo de memoria.
- Salida en consola estructurada y legible.

Consideraciones

El modelo representa un torneo realista manteniendo una estructura determinista. Se implementó:

- El ranking FIFA para ordenar a los equipos y definir los bombos.
- El promedio histórico de goles para establecer la expectativa de rendimiento.
- La distribución de Poisson para la generación probabilística de goles.
- Criterios de desempate estándar por puntos, diferencia de goles y goles a favor.

Estructura del programa y responsabilidades

El código fuente se organizó de esta manera:

1. **App:** app.cpp. El método App::iniciar() controla el flujo de ejecución principal.
2. **Repositorio:** repositorio.h/cpp. Se encarga de leer el CSV, almacenar datos históricos y exponer métodos de acceso para equipos, confederaciones, ranking y estadísticas.
3. **Torneo:** torneo.h/cpp. Tiene la lógica del sorteo, calendario, simulación de grupos, clasificación y fases eliminatorias.
4. **Simulador:** simulador.h/cpp. Prueba de manera independiente el modelo de cálculo de goles.
5. **Medidor:** medidor.h/cpp. Imprime las métricas de memoria e iteraciones.
6. **Equipo/Jugador/Grupo/Partido/TablaPosiciones:** Clases de dominio que modelan las entidades del torneo.
7. **arreglodinamico/matrizdinamica/listasimple:** Contenedores que se creó para el manejo de colecciones de datos.

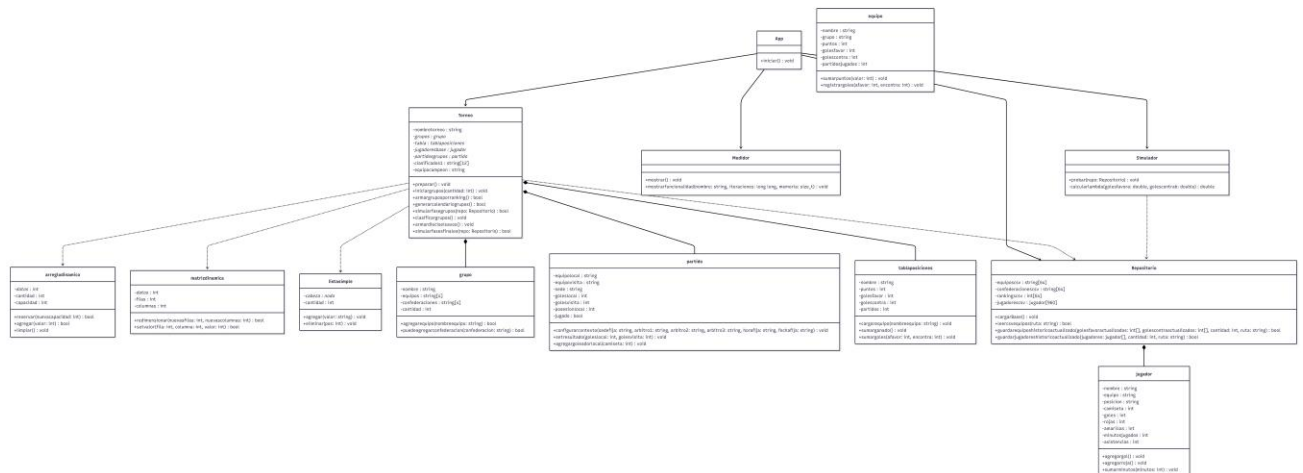
El código interactúa de manera que App coordina a Repositorio, Torneo, Simulador y Medidor; mientras que Torneo interactúa con Repositorio solo para consultar datos o guardar resultados.

Diagrama UML de clases (EVOLUCION)

Como se puede notar al comparar ambos diagramas, el diseño de las clases desde la primera entrega hasta el código final. El primer diagrama fue más que todo una idea inicial de cómo se relacionarían las cosas. Sin embargo, a la hora de programar, me di cuenta de que tener las clases tan amarradas unas con otras usando punteros directos (por ejemplo, que los equipos tuvieran punteros a los jugadores, o los grupos a los equipos) enredaba mucho el código y era muy fácil cometer errores de memoria.

Para solucionar esto, decidí simplificar las clases base. En lugar de usar punteros cruzados, ahora las entidades se referencian usando simplemente sus nombres en formato de texto (string). Además, opté por crear dos clases principales (Torneo y Repositorio) para que funcionen como administradores centrales de toda la información, quitándole esa carga a las demás clases. Por último, se tuvo que agregar clases nuevas que no estaban aun complementadas al inicio, como arreglodinamico y listasimple, para poder manejar las estructuras de datos de forma manual sin usar librerías externas.

El diagrama quedo en en un solo .png en el repositorio porque no quedaría bien acá en el informe la imagen dado alo extenso que puede ser(De igual forma es este)



Algoritmos esenciales y lógica de implementación

Carga inicial y preparación

El método `App::iniciar()` instancia los objetos principales y ejecuta:

- `repo.cargarBase()` para procesar el excel.
- `torneo.preparar()` para inicializar en memoria los 12 grupos vacíos.
- Una iteración sobre los equipos del repositorio invocando `torneo.cargarequipobase()`.
- Una iteración secundaria para extraer los jugadores con `repo.getjugador(i)` e ingresarlos mediante `torneo.cargarjugadorbase()`.

Sorteo de grupos con bombos y restricciones

El algoritmo en `Torneo::armargruposporranking()` sigue estos pasos:

1. Ordena los 48 índices según el ranking.
2. Estructura cuatro bombos de 12 equipos, asignando al anfitrión al primer bombo.
3. El programa asigna los equipos al azar dentro de los 12 grupos, validando paso a paso que al meter a cada uno no se rompa la regla de las confederaciones..
4. Si el algoritmo se queda bloqueado sin opciones válidas, se limpia el progreso actual y se repite la simulación del sorteo hasta encontrar una combinación exitosa..

Generación del calendario de la fase de grupos

`Torneo::generarcalendariogrupos()` programa las fechas garantizando un periodo de descanso adecuado entre partidos y limitando la cantidad de encuentros diarios. Si no encuentra una distribución viable, flexibiliza los parámetros automáticamente.

Simulación de la fase de grupos

`Torneo::simularfasegrupos()` procesa los 72 encuentros. Para cada partido:

- Identifica al equipo local y al visitante.
- Consulta sus estadísticas históricas de goles.
- Calcula el parámetro λ para ambos equipos.
- Genera el resultado final utilizando la distribución de Poisson.
- Asigna un porcentaje de posesión ponderado por el ranking.
- Actualiza las tablas de posiciones y los registros de los jugadores.

Clasificación de grupos

Torneo::clasificargrupos() evalúa el desempeño en cada grupo para seleccionar:

- Los 12 primeros.
- Los 12 segundos.
- Los 8 mejores terceros.

Para sacar a los mejores terceros, el algoritmo toma a los que ocuparon el tercer puesto de cada grupo, recorre sus datos usando arreglos fijos para comparar quién tiene mejores puntos o diferencia de goles, y clasifica a los 8 que sean superiores.

Armado de la llave de dieciseisavos

Torneo::armardieciseisavos() establece los cruces aplicando estas directrices:

- Los 4 peores segundos se enfrentan a los mejores primeros de los grupos I-L.
- Las selecciones restantes se emparejan asegurando que no se crucen equipos del mismo grupo.
- Si se detecta un cruce inválido, aplica un intercambio posicional de los visitantes.

La estructura final se almacena en memoria dinámica utilizando new partido[cantdieciseisavos].

Simulación de fases finales

La función Torneo::simularfasesfinales() procesa las rondas de eliminación directa. En caso de empate, la función resolverganadoreeliminacion() introduce una simulación de tiempo extra que otorga un ligero sesgo probabilístico al equipo mejor posicionado en el ranking.

Guardado de históricos

El flujo termina con Torneo::guardarhistoricoactualizado(), que itera sobre las estructuras en memoria y delega la escritura a la clase Repositorio para actualizar los archivos excel.

Contenedores

Se desarrollaron tres contenedores:

- arreglodinamico: sirve para ir agregando números en orden. Cuando se llena, usa la función reservar(int) para pedir más espacio y hacerse más grande, controlando toda la memoria a través de un puntero interno int* datos
- matrizdinamica: Básicamente imita el comportamiento de una matriz usando un solo arreglo lineal (de una sola dimensión). Para saber exactamente la ubicación de cada celda, hace el cálculo con la fórmula fila * columnas + columna

- **listasimple:** Se usó para manejar cadenas de texto de forma dinámica.

Problemas durante el desarrollo y cómo se solucionaron

El sorteo y la regla de las confederaciones: El problema al armar los grupos al azar era que a veces quedaban equipos de la misma región juntos, lo cual va contra las reglas del torneo. Para solucionarlo, se agregó una validación: si el código detecta un choque de confederaciones, el programa detiene el proceso, borra el intento actual y repite el sorteo desde cero hasta que los grupos queden bien repartidos.

Clasificar a los mejores terceros: Para definir quiénes pasan a la siguiente ronda como mejores terceros, se hizo una algo que guarda a estos equipos y los compara. Al recorrer el arreglo, el código revisa la cantidad de puntos y la diferencia de goles para poder separar a los 8 mejores del resto.

El manejo de la memoria dinámica: Como se tiene que administrar la memoria manualmente con punteros fue una de las partes más complejas para evitar que el programa se cerrara o cracheaba por errores. La solución fue crear destructores en todas las clases para asegurar que cualquier bloque de memoria reservado con un `new` se liberara correctamente con un `delete[]`. Además, se incluyeron validaciones para limpiar los punteros viejos antes de asignarles nueva información.

Los empates en las fases eliminatorias: En rondas como octavos o cuartos de final no pueden existir empates. Para solucionar esto en la simulación, se incluyó una función que actúa como un "tiempo extra". Para que el resultado no fuera completamente al azar, se agregó un cálculo que evalúa el ranking de ambos equipos, dándole una ventaja estadística más realista al mejor posicionado.

Generación del calendario de partidos: Para generar el calendario, el programa valida que no se supere el límite de partidos diarios y verifica que ningún equipo juegue dos días seguidos.